What is the problem? Generative modeling

- Data: $D = \{x_i\}_{i=1}^{N}$, where $x \in \mathcal{X}$ is high-dimensional (e.g., images).
- Assume $x_i \sim p_{\text{data}}(x)$.
- Goal: learn a probabilistic model $p_\theta(x)$ that approximates $p_{\text{data}}(x)$ and can generate new samples.
- Classical objective: maximum likelihood estimation (MLE).

Maximum likelihood objective

$$\theta^\star \in \arg\max_\theta \sum_{i=1}^{N} \log p_\theta(x_i).$$

Equivalently (up to constants), minimize negative log-likelihood:

$$\min_\theta \; -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(x_i).$$

For expressive models, directly specifying $p_\theta(x)$ is hard; latent-variable models provide a structured approach.

Latent-variable generative models: idea

Introduce latent variables $z \in \mathcal{Z}$ (typically lower-dimensional):

$$z \sim p_\theta(z), \qquad x \sim p_\theta(x \mid z).$$

Joint distribution:

$$p_\theta(x, z) = p_\theta(x \mid z)\, p_\theta(z).$$

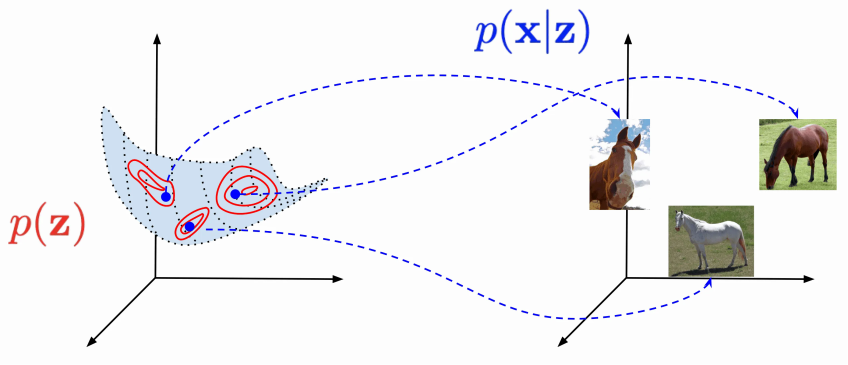
Marginal likelihood (the model for data):

$$p_\theta(x) = \int p_\theta(x \mid z)\, p_\theta(z)\, dz.$$

- $p_\theta(z)$: prior over latent space.
- $p_\theta(x \mid z)$: decoder / generative conditional distribution.

Why latent variables help

- High-dimensional x can be generated from lower-dimensional factors z.
- Complex, multimodal distributions over x can arise from simple priors over z.
- Latent space is often smoother and more interpretable (interpolation, clustering, traversals).



Naive Monte Carlo approximation

Since

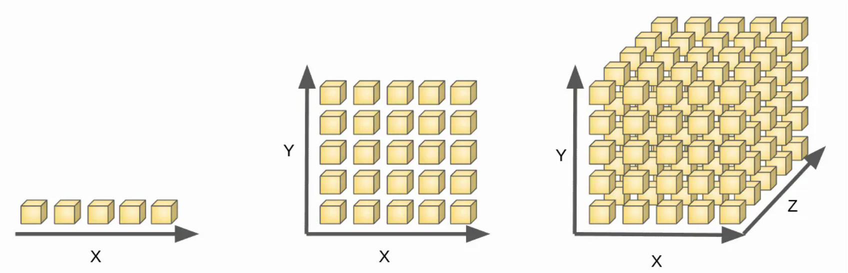
$$p_\theta(x) = \mathbb{E}_{z \sim p_\theta(z)}\left[p_\theta(x \mid z)\right],$$

a naive estimator is:

$$p_\theta(x) \approx \frac{1}{K}\sum_{k=1}^{K} p_\theta(x \mid z_k), \qquad z_k \sim p_\theta(z).$$

Problem:

- If $p_\theta(x \mid z)$ is sharply peaked, most z_k contribute almost nothing.
- Accurate Monte Carlo may require enormous K.



Importance sampling viewpoint

Rewrite the marginal likelihood using any proposal $q(z \mid x)$:

$$p_\theta(x) = \int p_\theta(x \mid z) p_\theta(z)\, dz = \int p_\theta(x \mid z) \frac{p_\theta(z)}{q(z \mid x)}\, q(z \mid x)\, dz.$$

Hence:

$$p_\theta(x) = \mathbb{E}_{z \sim q(z \mid x)} \left[p_\theta(x \mid z) \frac{p_\theta(z)}{q(z \mid x)} \right].$$

Monte Carlo estimator:

$$p_\theta(x) \approx \frac{1}{K} \sum_{k=1}^{K} p_\theta(x \mid z_k) \frac{p_\theta(z_k)}{q(z_k \mid x)}, \quad z_k \sim q(z \mid x).$$

If $q(z \mid x)$ concentrates where $p_\theta(x \mid z)$ is large, estimation improves.

ELBO: evidence lower bound

Start from:

$$\log p_\theta(x) = \log \int p_\theta(x,z)\,dz.$$

Multiply and divide by $q_\phi(z \mid x)$:

$$\log p_\theta(x) = \log \int q_\phi(z \mid x) \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\,dz = \log \mathbb{E}_{q_\phi(z|x)}\left[\frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right].$$

By Jensen's inequality:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x,z) - \log q_\phi(z \mid x)\right].$$

Define the ELBO:

$$\mathcal{L}_{\theta,\phi}(x) \;=\; \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x,z) - \log q_\phi(z \mid x)\right].$$

Standard ELBO decomposition

Using $p_\theta(x, z) = p_\theta(x \mid z) p_\theta(z)$:

$$\begin{aligned}
\mathcal{L}_{\theta,\phi}(x) &= \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] + \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(z)\big] - \mathbb{E}_{q_\phi(z|x)}\big[\log q_\phi(z \mid x)\big] \\
&= \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z)\big).
\end{aligned}$$

Two parts:

- Reconstruction (data fit): $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]$.
- Regularization: $-D_{\mathrm{KL}}(q_\phi(z \mid x) \| p_\theta(z))$.

ELBO gap: relation to the true log-likelihood

A key identity:

$$\log p_\theta(x) = \mathcal{L}_{\theta,\phi}(x) + D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big).$$

Since KL is nonnegative:

$$\log p_\theta(x) \geq \mathcal{L}_{\theta,\phi}(x).$$

Interpretation:

- Maximizing ELBO increases $\log p_\theta(x)$.
- At optimum, if $q_\phi(z \mid x) = p_\theta(z \mid x)$, the bound is tight.

Training objective over a dataset

Maximize the sum of ELBOs:

$$\max_{\theta,\phi} \sum_{i=1}^{N} \mathcal{L}_{\theta,\phi}(x_i) = \max_{\theta,\phi} \sum_{i=1}^{N} \left(\mathbb{E}_{q_\phi(z|x_i)}[\log p_\theta(x_i \mid z)] - D_{\mathrm{KL}}(q_\phi(z \mid x_i) \| p_\theta(z)) \right).$$

Stochastic optimization uses mini-batches and Monte Carlo estimates of expectations.

Interpretation of ELBO terms (key ideas)

- **Reconstruction term**:
$$\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]$$
encourages the decoder to make x likely given latents sampled from the encoder.
- **KL regularizer**:
$$D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z)\big)$$
encourages the encoded distribution to match the prior, which:
  - makes sampling $z \sim p(z)$ produce plausible x,
  - prevents arbitrary, disconnected latent codes.

VAE architecture: encoder + decoder

VAE combines:

- **Encoder** (inference network): $q_\phi(z \mid x)$.
- **Decoder** (generative network): $p_\theta(x \mid z)$.
- **Prior**: $p_\theta(z)$ (often fixed as $\mathcal{N}(0, I)$).

This is "amortized" inference: one neural network outputs parameters of $q_\phi(z \mid x)$ for any x.

The gradient problem: sampling inside the encoder

We need gradients of

$$\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]$$

w.r.t. ϕ.

But z is sampled from $q_\phi(z \mid x)$, which depends on ϕ (have you ever taken gradient by parameter of distribution??).

Naively, $\nabla_\phi \mathbb{E}_{q_\phi}[\cdot]$ is nontrivial (sampling breaks standard backprop).

Solution: **reparameterization trick** for continuous latents (notably Gaussians).

Reparameterization trick (Gaussian case)

If

$$q_\phi(z \mid x) = \mathcal{N}\left(\mu_\phi(x), \mathrm{diag}(\sigma_\phi^2(x))\right),$$

sample by:

$$\varepsilon \sim \mathcal{N}(0, I), \qquad z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon.$$

Then

$$\mathbb{E}_{z\sim q_\phi(z|x)}[f(z)] = \mathbb{E}_{\varepsilon\sim\mathcal{N}(0,I)}\left[f\left(\mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon\right)\right],$$

which is differentiable w.r.t. ϕ through μ_ϕ, σ_ϕ.

Monte Carlo estimate of ELBO (per sample)

With reparameterization, using K noise samples ε_k:

$$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{\varepsilon\sim\mathcal{N}(0,I)}\big[\log p_\theta(x \mid z(\varepsilon))\big] - D_{\mathrm{KL}}(q_\phi(z \mid x)\|p(z)),$$

approximate:

$$\mathcal{L}_{\theta,\phi}(x) \approx \frac{1}{K}\sum_{k=1}^{K} \log p_\theta\big(x \mid z_k\big) - D_{\mathrm{KL}}(q_\phi(z \mid x)\|p(z)), \quad z_k = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon_k.$$

Commonly, $K = 1$ works well with mini-batches.

Connection to classical autoencoders

Deterministic autoencoder:

$$z = f_\phi(x), \qquad \hat{x} = g_\theta(z),$$

with reconstruction loss (e.g., $\|x - \hat{x}\|^2$).
VAE differences:

- encoder outputs a *distribution* $q_\phi(z \mid x)$, not a point,
- decoder is probabilistic $p_\theta(x \mid z)$,
- KL term shapes z to follow a chosen prior, enabling unconditional sampling.

Typical training loop (mini-batch SGD)

For a mini-batch $\{x_b\}_{b=1}^{B}$:

1. Encoder forward: compute $\mu_b = \mu_\phi(x_b)$ and $\log \sigma_b^2 = \log \sigma_\phi^2(x_b)$.
2. Sample noise: $\varepsilon_b \sim \mathcal{N}(0, I)$.
3. Reparameterize: $z_b = \mu_b + \sigma_b \odot \varepsilon_b$.
4. Decoder forward: compute parameters of $p_\theta(x_b \mid z_b)$.
5. Compute:

$$\widehat{\mathcal{L}} = \frac{1}{B} \sum_{b=1}^{B} \log p_\theta(x_b \mid z_b) - \frac{1}{B} \sum_{b=1}^{B} D_{\mathrm{KL}}\big(q_\phi(z \mid x_b) \| p(z)\big).$$

6. Update θ, ϕ by ascending $\widehat{\mathcal{L}}$ (or descending $-\widehat{\mathcal{L}}$).

Generation after training

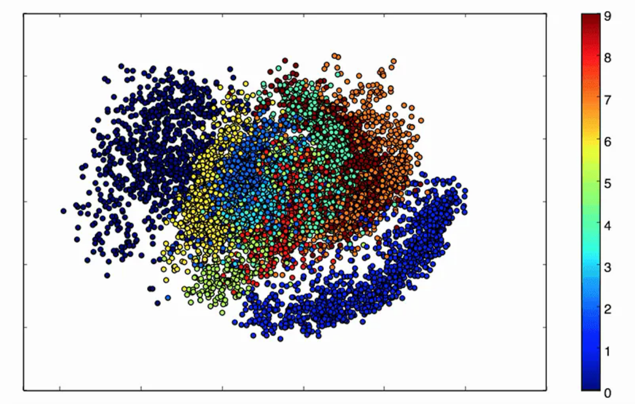
After training, to generate a new sample:

$$z \sim p(z) \quad (\text{often } \mathcal{N}(0, I)), \qquad x \sim p_\theta(x \mid z).$$

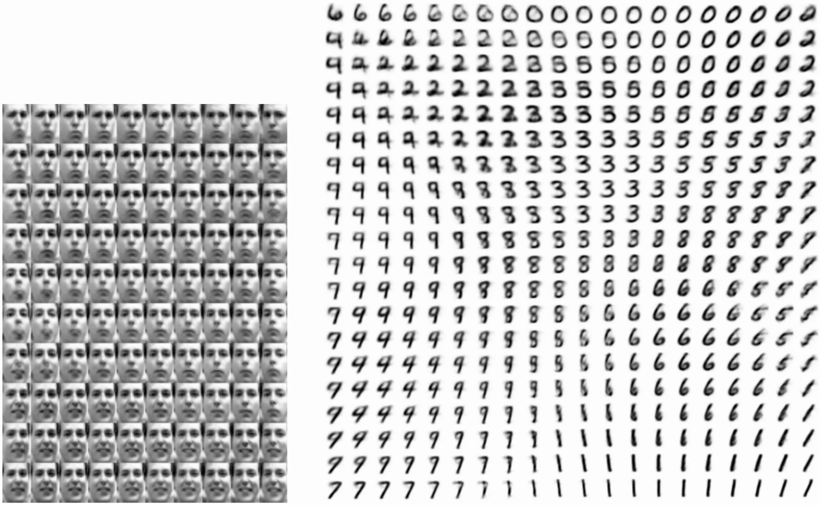
Latent space properties (key ideas)

Because $q_\phi(z \mid x)$ is regularized toward $p(z)$:

- nearby points in latent space often decode to semantically similar outputs,
- interpolation is meaningful: for $z(t) = (1 - t)z_0 + tz_1$
- latent space can reflect class structure (clusters/regions) when data has discrete modes.



Manifold



(a) Learned Frey Face manifold

(b) Learned MNIST manifold

Why the KL term matters for sampling

If we trained only reconstruction (like a deterministic autoencoder):

- encoded z could occupy an arbitrary set in $\mathbb{R}^M$,
- sampling $z \sim \mathcal{N}(0, I)$ would likely land outside that set,
- decoded samples would be garbage.

KL regularization enforces:

$$q_\phi(z \mid x) \approx p(z),$$

so that sampling from $p(z)$ produces latents similar to those seen during training.

Conditional generation: motivation

Sometimes we want to generate not from the overall distribution $p(x)$, but from a conditional distribution $p(x \mid y)$ for some attribute/label y:

- class label (digits, categories),
- style, domain, speaker identity,
- any conditioning information available at train/test time.

Solution: **Conditional VAE (CVAE)**.

Conditional VAE: model and inference

Introduce condition y and define:

$$p_\theta(z \mid y), \qquad p_\theta(x \mid z, y), \qquad q_\phi(z \mid x, y).$$

The conditional marginal likelihood:

$$p_\theta(x \mid y) = \int p_\theta(x \mid z, y) p_\theta(z \mid y)\, dz.$$

ELBO for CVAE:

$$\mathcal{L}_{\theta,\phi}(x, y) = \mathbb{E}_{q_\phi(z|x,y)}[\log p_\theta(x \mid z, y)] - D_{\mathrm{KL}}\big(q_\phi(z \mid x, y) \,\|\, p_\theta(z \mid y)\big).$$

CVAE: sampling

To generate conditioned on y:

$$z \sim p_\theta(z \mid y), \qquad x \sim p_\theta(x \mid z, y).$$

Common choices:

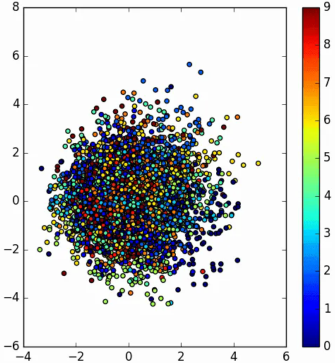
- $p_\theta(z \mid y) = \mathcal{N}(0, I)$ for all y, but feed y only to the decoder and encoder;
- or a class-dependent prior $p_\theta(z \mid y) = \mathcal{N}(\mu_y, \Sigma_y)$ (or NN-parameterized).

Why CVAE can separate classes better than a vanilla VAE

In a **vanilla VAE**, all data classes share one latent prior: $p(z) \sim \mathcal{N}(0, I)$.

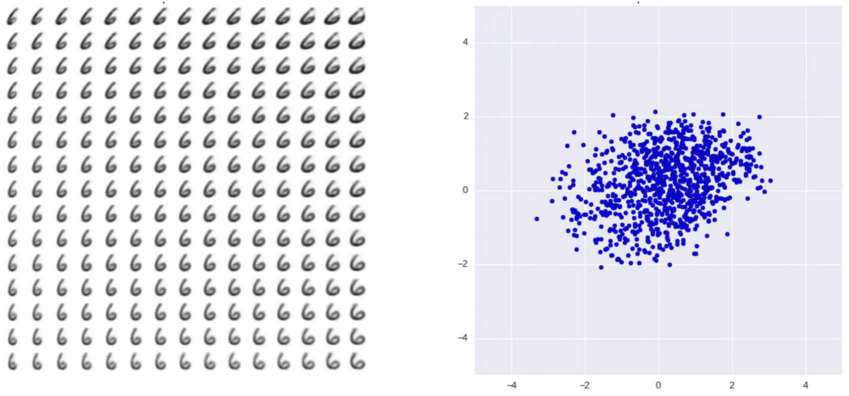
In a **Conditional VAE (CVAE)**, we condition the prior on the label y and effectively give each class its own latent space:

$$p(z \mid y) \sim \mathcal{N}(0, I) \qquad \text{for each } y \in \{0, \ldots, 9\}.$$



Why CVAE can separate classes better than a vanilla VAE

As a result, the model does not need to pack all digits into one shared $p(z)$; it can learn *class-specific* latent structure, which often yields cleaner class separation.



ELBO recap and the full picture

For (unconditional) VAE:

$$\log p_\theta(x) \geq \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{data fit / reconstruction}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x) \| p(z))}_{\text{latent regularization}}.$$

Training:

$$\max_{\theta,\phi} \sum_{i=1}^{N} \mathcal{L}_{\theta,\phi}(x_i).$$

After training:

$$z \sim p(z) \Rightarrow x \sim p_\theta(x \mid z).$$